Final Project Requirements Plan

Guitar Chord Progression Generator (Drafter web app)

Project Summary

The app lets a user pick a key, a mood, a progression length, and options such as 7th chords and beginner mode. It builds a chord progression that matches those choices and shows a guitar fingering diagram for every chord. Users can transpose the progression up or down, save progressions, rename or delete them, open a chord to see its notes, hear it, and see five ways to play it (the CAGED shapes), and read a Mood Guide. In beginner mode, the app avoids barre chords by suggesting a capo or an alternate pattern.

This document lists every requirement from the final project assignment and explains how the finished project meets it.

Site Functionality Requirements

Requirement	Minimum	How this project meets it
At least 7 routes	7	21 routes. Setup and result: index, generate, show_result, transpose_up, transpose_down, reset_transpose. Saving: save_page, save_progression, cancel_save. Chord detail and moods: chord_detail, open_mood_guide, view_mood_guide, choose_mood. Saved progressions: open_saved, view_saved, view_progression, confirm_delete, delete_progression, rename_page, rename_progression, cancel_rename.
At least 5 pages	5	9 pages: Setup, Result, Save, Chord Detail, Mood Guide, Saved Progressions, one Saved Progression, Delete confirmation, and Rename. The five main pages are Setup, Result, Chord Detail, Saved Progressions, and Mood Guide.
State dataclass with at least 4 fields	4	State has 12 fields: key, mood, length, use_sevenths, beginner_mode, transpose_steps, current_progression, saved_progressions, capo, pattern, progression_key, progression_mood. key, mood, length, use_sevenths, and beginner_mode remember what the setup page shows. The other fields describe the progression that is on screen.
A field that is a list (or dictionary) of class instances	1	current_progression: list[Chord] and saved_progressions: list[Progression].

Requirement	Minimum	How this project meets it
A second dataclass	1	Chord (name, root, quality, frets, is_barre, numeral) and Progression (name, mood, key, chords, capo). CagedShape (letter, start_fret, frets) is a third dataclass, used on the chord detail page.
4 state fields meaningfully modified in routes (not constants)	4	generate: sets key, mood, length, use_sevenths, beginner_mode, transpose_steps, capo, pattern, progression_key, progression_mood, and current_progression. transpose_up, transpose_down, reset_transpose: change transpose_steps and rebuild current_progression (and the capo in beginner mode). save_progression: appends to saved_progressions. delete_progression: removes from it. rename_progression: changes a saved progression's name. That is far more than four fields.
At least 3 input fields	3	SelectBox: key, mood, length. CheckBox: include 7th chords, beginner mode (no barre chords). TextBox: progression name when saving, and new name when renaming.
At least 3 meaningful if statements	3	Beginner mode on and the progression needs barre chords: look for a capo fret, then for an alternate pattern, and show an error only if neither works (generate, change_transpose). Name is empty, or already used (capital letters ignored): reject the save or rename with a message (save_progression, rename_progression). Saved list is empty: show 'Nothing saved yet' (make_saved_page). Minor key: build from the relative major (resolve_key). 7ths box checked: use seventh chords (choose_quality). Capo above 0: show 'sounds as' names and the capo line. transpose_steps not 0: show the reset button. A chord or progression name that no longer exists: show a message instead of failing (chord_detail, view_progression).
A loop over an attribute of the list of class instances	1	make_result_page loops over state.current_progression to draw each chord card. make_saved_page loops over saved_progressions to list the saved items, and each one's chords. name_is_taken and find_position loop over saved_progressions to check names. draw_dots and draw_markers loop over each chord's frets to draw each string of a diagram.

Requirement	Minimum	How this project meets it
Legitimate purpose or functionality	-	Helps singer-songwriters find fresh chord ideas for a mood, and helps beginner guitarists learn which chords work together and how to play them without barre chords.
No global variables (global constants are fine)	-	Only uppercase constants at module level, such as NOTES, MAJOR_SCALE_STEPS, MOOD_TEMPLATES, OPEN_CHORD_FINGERINGS, CHORD_FINGERINGS, CAGED_SHAPES, MOOD_COLORS, and CAPO_MAX. All changing data lives in State, which is passed into and returned from routes.
Must use the Drafter library	-	The whole app is built with @route, Page, State, SelectBox, CheckBox, TextBox, Button, Argument, Header, Div, Row, HorizontalRule, SVG, Tone, Melody, and start_server.
Advanced techniques documented in code	-	Modular arithmetic on a 12-note list for transposition ((index + steps) % 12) in transpose_note. Barre chord generation by sliding E-shape and A-shape templates up the neck (shift_shape, barre_fingering). CAGED shapes: each of the five shapes is slid so its root lands on the chord's root, using (root - shape_root) % 12 (caged_start_fret, caged_frets). Capo search: shift every chord down by a capo fret and keep the lowest fret that leaves no barre chord (find_capo_for_chords, find_beginner_options). SVG markup built as a string from fret data (draw_diagram) and a drawn guitar (draw_guitar). Sound: Tone and Melody components play each note and a strum of the chord on the detail page. Each has a docstring or comment explaining what it does and why it works.
Extra creativity (flexibility on requirements)	-	Guitar fingering diagrams drawn as SVG, transposing, beginner mode with capo suggestions and alternate patterns, all five CAGED shapes for every chord, sound on the chord detail page, a Mood Guide that can start a progression, Roman numerals under each chord, mood colors, and rename and delete with confirmation.

Note: missing more than two requirements earns no credit. Missing the second dataclass would also mean missing the list of instances and the loop over it, so these three were the first priority.

Code Requirements

Requirement	How this project meets it
Program runs and deploys	The site is deployed on GitHub Pages at https://mcataniajr.github.io/guitar-progression-generator/. The deployment workflow also runs a mypy type check on every push.
Good variable names	Descriptive names such as scale_notes, current_progression, find_beginner_options, and progression_name instead of single letters.
Unit tests	Pure helpers are tested with assert_equal: transpose_note, get_scale, relative_major, parse_key, numeral_to_chord, build_progression, draw_diagram, shift_chords, find_capo_for_chords, and more. Every route has at least one test using assert_has, assert_state, or assert_equal. Extra checks cover every key and mood, all 300 CAGED shapes, and every beginner-mode case (capo, alternate pattern, and the error message). The app uses no random choices, so every route can be tested.
Note functions that can't be unit tested	make_*_page helpers are checked with assert_has on the page text. draw_guitar only returns drawing text and is checked for its markup.
Lines no longer than 120 characters	Checked by script: no line in main.py is over 120 characters. Long SVG strings and dictionary rows are split across lines.
Types in function headers	Every function has parameter and return types, such as def transpose_note(note: str, steps: int) -> str. Page-building lists are typed list[Any], and mypy passes in the deployment workflow.
Docstrings as the first line of each body	Every function starts with a docstring describing what it does.
Decompose complicated functions	Music logic is split into small helpers (get_scale, numeral_to_chord, build_progression, transpose_note, lookup_fingering, caged_shapes, find_capo_for_chords). Drawing is split into draw_grid, draw_markers, draw_barre, and draw_dots. Routes call helpers and update State, and each page has a make_*_page function.

Repository Requirements

Requirement	How this project meets it
Planning document uploaded to the repository	This document and the brainstorming and design worksheets go in the planning folder.
Website deployed without error	Deployed through GitHub Pages: https://mcataniajr.github.io/guitar-progression-generator/
Site information set with set_site_information(...) in main.py	Called at the bottom of main.py with the author, a description, the sources (Drafter docs and Claude), the planning folder, and links to the repository and the live site.

Requirement	How this project meets it
Descriptive commit messages	Messages such as 'Fix mypy type errors in main.py' and 'Add CAGED shapes, UI polish, and capo-based beginner mode' instead of 'Update main.py'.
Video linked in the repository	To do: record a video of the deployed site (5 minutes or less, address bar visible) covering choosing a key and mood, the progression and diagrams, transposing, beginner mode, saving, renaming, deleting, the chord detail page, and the mood guide. Link it in the README.
All AI conversations uploaded	Every conversation with Claude used for brainstorming, planning, and coding goes in the ai-conversations folder.

Design Decisions That Support the Requirements

Topic	Decision
Key handling	NOTES uses sharps internally. Flat keys (Eb, Bb) are converted at input and display only. Black keys are shown as C#, Eb, F#, G#, Bb. The key list is A, Bb, B, C ... G# (major), then Am ... G#m.
Minor keys	Built from the relative major (3 semitones up). Minor moods start on the vi chord, so A minor begins on Am, not C.
Moods (5)	Joyful, Peaceful, Sorrowful, Mysterious, Bluesy. Each maps to one or two Roman numeral patterns (MOOD_TEMPLATES). Bluesy uses the 12-bar blues. Each mood has a description and an accent color.
Transposition	transpose_steps counts half steps up or down. The chords themselves are moved with modular arithmetic, so a transposed progression can be saved and reset.
Chord fingerings	OPEN_CHORD_FINGERINGS stores the open chords. Barre chords are generated from E-shape and A-shape templates. CHORD_FINGERINGS is built once from both for every root and quality (60 chords).
Beginner mode	A progression that needs barre chords gets the lowest capo fret (0 to 7) that makes every chord an open shape. If the main pattern cannot work, the mood's other patterns are tried. An error is shown only if nothing works. Without 7th chords this covers 104 of 120 key and mood choices, and with 7ths it covers all 120.
CAGED shapes	The chord detail page shows five ways to play the chord, one per CAGED shape, ordered from low to high on the neck. With a capo, they are for the chord that sounds.
Diagrams and sound	Drafter's SVG component draws strings, frets, dots, and barre bars from each fret list. Tone and Melody play the notes on the chord detail page.
Saved progressions	Identified by name. Names cannot repeat (capital letters ignored). Each saved progression keeps its own key, mood, and capo.

Topic	Decision
Drafter inputs	Drafter sends every input on a page to the route a button points to. Routes reachable from pages with inputs (setup, save, rename) therefore accept the matching parameters, and the shared navigation row appears only on pages without inputs.

State and dataclasses

- Chord: name (str), root (str), quality (str), frets (list[int]), is_barre (bool), numeral (str, the Roman numeral, ignored when chords are compared).
- Progression: name (str), mood (str), key (str), chords (list[Chord]), capo (int).
- CagedShape: letter (str), start_fret (int), frets (list[int]).
- State: key, mood, length, use_sevenths, beginner_mode, transpose_steps, current_progression, saved_progressions, capo, pattern, progression_key, progression_mood.

Routes

- index: shows the setup page. generate: reads the form, builds the progression, and handles beginner mode.
- show_result: displays the progression and diagrams. transpose_up, transpose_down, reset_transpose: move the key and rebuild.
- save_page, save_progression, cancel_save: name and save the current progression.
- chord_detail: one chord with its notes, sound, and five CAGED shapes.
- open_mood_guide, view_mood_guide, choose_mood: explain each mood and start a progression from one.
- open_saved, view_saved, view_progression: list and open saved progressions.
- confirm_delete, delete_progression: delete after a confirmation page.
- rename_page, rename_progression, cancel_rename: rename without repeating a name.

Pages

- Setup, Result, Save, Chord Detail, Mood Guide, Saved Progressions, Saved Progression, Delete confirmation, Rename.

Schedule

Week	Plan	Status
Week 10: Brainstorming	Choose the idea, finish the goal, IPO model, and data abstraction answers, and confirm design decisions.	Done
Week 11: Design	Finalize the page sketches, State and dataclass fields, chord dictionary format, mood templates, and the planning document.	Done
Week 12: Development and deployment	Build helpers and tests first, then routes, then pages and diagrams. Deploy early.	Done: built, tested, and deployed

Week	Plan	Status
Week 13: User testing	Have peers try the site. Pay attention to whether the minor-key handling, the transpose buttons, and beginner mode make sense.	To do
Week 14: Updates based on feedback	Fix what testers found, finish the video, upload AI conversations, and write the reflection.	To do

Risks and How They Were Handled

Risk	Plan and result
Scope grows too large	Finished the core path (generate, display, transpose, save) before any extras.
Diagrams are hard to draw	Started with a small set of chords and a simple SVG, then added X markers, barre bars, and larger diagrams.
Minor key bug	Dedicated tests for minor keys such as Am, Em, and Dm.
Beginner mode rejects most choices	Added capo suggestions and alternate patterns; the remaining cases get a clear error message.
A progression shown with the wrong key or mood	The current progression keeps its own key and mood (progression_key, progression_mood), separate from the setup page's choices.
Missing too many requirements	Missing more than two earns no credit, so the second dataclass, the list of instances, and the loop over it were secured first.